



Optimism: Upgrade 19

Security Review

Cantina Managed review by:

Oxlcinddeath, Lead Security Researcher

R0bert, Lead Security Researcher

May 23, 2026

Contents

1	Introduction	2
1.1	About Cantina	2
1.2	Disclaimer	2
1.3	Risk assessment	2
1.3.1	Severity Classification	2
2	Security Review Summary	3
3	Findings	4
3.1	Medium Risk	4
3.1.1	L2CM drops X Layer liquidity minters	4
3.1.2	OPCM utils allows unsupported v5 initializers	4
3.1.3	Separate FeeVault route setters can misroute accrued fees	5
3.2	Low Risk	6
3.2.1	<code>assertValidStateRegistry</code> is missing a check on the respected game state	6
3.2.2	<code>assertValidSharedLockbox</code> does not implement an explicit <code>systemConfig</code> check	7
3.2.3	<code>ORACLE</code> checks are missing in <code>assertValidMipsVm</code>	7
3.2.4	Renounced SystemConfig owner blocks normal OPCM upgrades	8
3.2.5	Guardian address check should be added to the <code>assertValidSuperchainConfig</code> function	8
3.2.6	Missing check on bridge version <code>assertValidOptimismMintableERC20Factory</code>	8
3.2.7	Validator misses portal <code>AnchorStateRegistry</code> check	9
3.2.8	Validator omits critical SystemConfig checks	9
3.2.9	Assertion checks should ensure that owners of the <code>disputeGameFactory</code> and <code>proxyAdmin</code> are sound	10
3.2.10	Unauthorized root-block challenge wins bond	10
3.3	Informational	10
3.3.1	ASR cutovers can leave old games valid	10
3.3.2	Portal misses chain-ID check for non-super game roots	11
3.3.3	Add check on <code>START_BLOCK_SLOT</code> in <code>assertValidSystemConfig</code>	12

1 Introduction

1.1 About Cantina

Cantina is a security services marketplace that connects top security researchers and solutions with clients. Learn more at cantina.xyz

1.2 Disclaimer

Cantina Managed provides a detailed evaluation of the security posture of the code at a particular moment based on the information available at the time of the review. While Cantina Managed endeavors to identify and disclose all potential security issues, it cannot guarantee that every vulnerability will be detected or that the code will be entirely secure against all possible attacks. The assessment is conducted based on the specific commit and version of the code provided. Any subsequent modifications to the code may introduce new vulnerabilities that were absent during the initial review. Therefore, any changes made to the code require a new security review to ensure that the code remains secure. Please be advised that the Cantina Managed security review is not a replacement for continuous security measures such as penetration testing, vulnerability scanning, and regular code reviews.

1.3 Risk assessment

Severity level	Impact: High	Impact: Medium	Impact: Low
Likelihood: high	Critical	High	Medium
Likelihood: medium	High	Medium	Low
Likelihood: low	Medium	Low	Low

1.3.1 Severity Classification

The severity of security issues found during the security review is categorized based on the above table. Critical findings have a high likelihood of being exploited and must be addressed immediately. High findings are almost certain to occur, easy to perform, or not easy but highly incentivized thus must be fixed as soon as possible.

Medium findings are conditionally possible or incentivized but are still relatively likely to occur and should be addressed. Low findings are a rare combination of circumstances to exploit, or offer little to no incentive to exploit but are recommended to be addressed.

Lastly, some findings might represent objective improvements that should be addressed but do not impact the project's overall security (Gas and Informational findings).

2 Security Review Summary

Optimism is a fast, stable, and scalable L2 blockchain built by Ethereum developers, for Ethereum developers. Built as a minimal extension to existing Ethereum software, Optimism's EVM-equivalent architecture scales your Ethereum apps without surprises. If it works on Ethereum, it works on Optimism at a fraction of the cost.

From May 4th to May 15th the Cantina team conducted a review of [optimism](#) on commit hash [7cbcb58f](#). The team identified a total of **16** issues:

Issues Found

Severity	Count	Fixed	Acknowledged
Critical Risk	0	0	0
High Risk	0	0	0
Medium Risk	3	0	0
Low Risk	10	0	1
Gas Optimizations	0	0	0
Informational	3	0	1
Total	16	0	2

3 Findings

3.1 Medium Risk

3.1.1 L2CM drops X Layer liquidity minters

Severity: Medium Risk

Context: `L2ContractsManager.sol#L224-L245`

Description: `_loadFullConfig` has a special fallback for the documented X Layer `LiquidityController` fork when `owner()` reverts, but it snapshots only the owner fallback plus token name and symbol. It does not snapshot existing authorized `minters` and does not require any explicit reauthorization data.

`_apply` then installs the OP Stack `LiquidityController` and calls `initialize` with only those three fields. The new implementation inherits `OwnableUpgradeable` before `minters`, so its `minters` mapping is at slot 101. A no-`owner()` fork that authorizes minters from its own first mapping stores those authorizations at slot 1. Therefore a successful L2CM upgrade preserves the owner and metadata while silently leaving previously authorized minters unreadable by the new implementation.

With `SYS_FEATURE__CUSTOM_GAS_TOKEN=true`, the old minter is authorized before `_executeUpgrade()`, `upgraded.minters(minter)` is false after the upgrade and `mint` reverts with `LiquidityController_Unauthorized` until the proxy admin owner manually reauthorizes the minter. Existing bridge, migration or operational native-asset liquidity minters can therefore halt after the upgrade even though the upgrade transaction itself succeeds.

Recommendation: Make minter preservation explicit for the X Layer compatibility migration. Either fail preflight unless the upgrade bundle includes the required minter reauthorization list and post-upgrade validation, or add a fork-specific migration that copies the known minter slots or authorizations before native-asset minting and burning resume.

3.1.2 OPCM utils allows unsupported v5 initializers

Severity: Medium Risk

Context: `OPContractsManagerUtils.sol#L343-L359`

Description: `OPContractsManagerUtils.upgrade` tries to reject OpenZeppelin v5 `Initializable` contracts by checking the caller-supplied slot and the target proxy's ERC-7201 initializer slot before the final upgrade. That only detects proxies that already have v5 initializer state.

A proxy that currently uses the v4 layout has an empty ERC-7201 slot, so it passes the guard. If the incoming implementation uses the v5 initializer layout, `upgradeAndCall` then delegates to its initializer and writes `_initialized` into the ERC-7201 slot. The helper has now installed an implementation that its own guard says is unsupported.

Consequently, the next standard OPCM upgrade for that proxy reverts before it can repair or replace the implementation, because the same guard now sees the nonzero ERC-7201 slot. A single release that accidentally includes a v5-style implementation can leave the proxy outside the normal OPCM upgrade mechanism until a custom recovery helper or manual storage fix is used.

Example:

1. A proxy currently points to a v4-style implementation.
2. The proxy's v5 ERC-7201 initializer slot is empty.
3. A release accidentally provides a v5-style implementation.
4. `OPContractsManagerUtils.upgrade` checks the v5 slot and sees `bytes32(0)`, so it does not revert.
5. The helper clears the old v4 initializer byte.
6. The helper calls `upgradeAndCall` with the new implementation.

7. The new implementation's initializer writes to the v5 ERC-7201 initializer slot.
8. The upgrade succeeds, even though OPCM says v5 initializer state is unsupported.
9. The next standard OPCM upgrade checks the same v5 slot, now sees a nonzero value, and reverts with `OPContractsManagerUtils_0Zv5InitializableUnsupported`.

Recommendation: Make the unsupported initializer-layout rule apply to the incoming implementation before it is installed. If OPCM intends to reject v5 initializers, require release implementations to expose a supported-layout marker or enforce an implementation codehash/metadata allowlist that `upgrade` can check before `upgradeAndCall`.

If v5 implementations should be allowed, support that layout deliberately instead: clear the ERC-7201 `_initialized` field and check the v5 `_initializing` flag.

3.1.3 Separate FeeVault route setters can misroute accrued fees

Severity: Medium Risk

Context: [FeeVault.sol#L113-L163](#)

Description: `FeeVault` stores `recipient` and `withdrawalNetwork` as separate fields and the proxy admin owner must update them with separate calls. `withdraw` is permissionless and always withdraws the full vault balance to whatever pair is stored at that moment.

This creates an unsafe intermediate state when an operator changes a vault from an L1 recipient to an L2 recipient or the reverse. If the owner updates `withdrawalNetwork` first, anyone can call `withdraw` before `recipient` is updated and send the full balance to the old recipient address on the new network. If the owner updates `recipient` first, anyone can create a withdrawal to the new recipient address on the old network.

Both the old and new configurations can be valid. The problem is the transient cross-product created by independent setters. This matters for contract recipients such as Safes, treasuries or network-specific payout contracts because the same 20-byte address may not be controlled on the other network. A public caller can therefore turn a routine route migration into a full-balance transfer or L1 withdrawal message to an address that the intended recipient cannot recover from.

Recommendation: Add an atomic route update function that sets `recipient` and `withdrawalNetwork` together, and deprecate separate route changes for cross-network migrations. For example:

```
function setWithdrawalRoute(address _newRecipient, Types.WithdrawalNetwork
↪ _newWithdrawalNetwork) external {
    _assertOnlyProxyAdminOwner();
    require(_newRecipient != address(0), "FeeVault: zero recipient");

    address oldRecipient = recipient;
    Types.WithdrawalNetwork oldWithdrawalNetwork = withdrawalNetwork;

    recipient = _newRecipient;
    withdrawalNetwork = _newWithdrawalNetwork;

    emit RecipientUpdated(oldRecipient, _newRecipient);
    emit WithdrawalNetworkUpdated(oldWithdrawalNetwork, _newWithdrawalNetwork);
}
```

If the separate setters must remain for compatibility, document that operators must first reduce exposure by withdrawing the vault balance or raising the minimum above the current balance, then execute the route change atomically through a batching wallet or migration contract.

3.2 Low Risk

3.2.1 `assertValidStateRegistry` is missing a check on the respected game state

Severity: Low Risk

Context: `StandardValidatorUtils.sol#L344`

Description: In the `initialize` function of the `AnchorStateRegistry`, there's a series of variables that are set, that are not checked in the associated valid assertion functions.

```
/// @notice Initializes the contract.
/// @param _systemConfig The address of the SystemConfig contract.
/// @param _disputeGameFactory The address of the DisputeGameFactory contract.
/// @param _startingAnchorRoot The starting anchor root.
/// @param _startingRespectedGameType The starting respected game type.
function initialize(
    ISystemConfig _systemConfig,
    IDisputeGameFactory _disputeGameFactory,
    Proposal memory _startingAnchorRoot,
    GameType _startingRespectedGameType
)
{
    external
    reinitializer(1)
    {
        // Initialization transactions must come from the ProxyAdmin or its owner.
        _assertOnlyProxyAdminOrProxyAdminOwner();

        // Now perform initialization logic.
        systemConfig = _systemConfig;
        disputeGameFactory = _disputeGameFactory;
        respectedGameType = _startingRespectedGameType;

        // If the starting anchor root is changing and an anchor game exists, verify the new
        // root
        // is ahead and clear the anchor game so getAnchorRoot() returns the new
        // startingAnchorRoot.
        bool rootChanged = _startingAnchorRoot.root.raw() != startingAnchorRoot.root.raw()
            || _startingAnchorRoot.l2SequenceNumber != startingAnchorRoot.l2SequenceNumber;
        if (rootChanged && address(anchorGame) != address(0)) {
            (, uint256 currentSeqNum) = getAnchorRoot();
            if (_startingAnchorRoot.l2SequenceNumber <= currentSeqNum) {
                revert AnchorStateRegistry_InvalidAnchorGame();
            }
            anchorGame = IDisputeGame(address(0));
        }
        startingAnchorRoot = _startingAnchorRoot;

        // Set the retirement timestamp to the current timestamp the first time the contract
        // is
        // initialized. This was originally done in U16a to guarantee that all games created
        // before
        // the initialization of the new AnchorStateRegistry would be retired. This is no
        // longer
        // required behavior but it's useful to maintain common behavior across all new and
        // old
        // AnchorStateRegistry instances. We don't set the retirement timestamp if already
        // set to
        // avoid retiring games when re-initializing the contract during an upgrade.
        if (retirementTimestamp == 0) {
            retirementTimestamp = uint64(block.timestamp);
        }
    }
}
```

As the assertion function is meant to check proper deployment, the following are missing: `respectedGameType`, `retirementTimestamp`, `startingAnchorRoot` and `anchorGame`. With respect

to U19-specific upgrades, it's especially important for the respected game type to be set correctly, and for the assertion function to fail.

Recommendation: Add a check to check the correct `respectedGameType` , `retirementTimestamp` , `anchorGame` , and `startingAnchorRoot` .

3.2.2 `assertValidSharedLockbox` does not implement an explicit `systemConfig` check

Severity: Low Risk

Context: `OPContractsManagerMigrationValidator.sol#L375-L376`

Description: The `ETHLockbox` contract implements the `initialize` function, which sets the `systemConfig` and the appropriate portals. Many other functions have implicit checks on the system config, by way of querying a contract in the config, and checking data. However, there does not seem to be any explicit check that the lockbox has been setup with the correct system config - even in the parent contract where the function is called in `validateMigrationWithOverrides` . As such, it is possible that an `ETHLockbox` is deployed, using a different config, and it will not be caught by these assertion functions.

```
/// @notice Constructs the ETHLockbox contract.
constructor() ReinitializableBase(1) {
    _disableInitializers();
}

/// @notice Initializer.
/// @param _systemConfig The address of the SystemConfig contract.
/// @param _portals The addresses of the portals to authorize.
/// @dev Note: Multiple chains can share an ETHLockbox contract. In this case, all
→ SystemConfig
/// contracts will point to the same pause identifier (the lockbox itself).
→ Therefore, it
/// doesn't matter which SystemConfig is used here as long as it belongs to one of
→ the
/// chains that share the lockbox.
function initialize(
    ISystemConfig _systemConfig,
    IOptimismPortal[] calldata _portals
)
    external
    reinitializer(initVersion())
{
    // Initialization transactions must come from the ProxyAdmin or its owner.
    _assertOnlyProxyAdminOrProxyAdminOwner();

    // Now perform initialization logic.
    systemConfig = _systemConfig;
    for (uint256 i; i < _portals.length; i++) {
        _authorizePortal(_portals[i]);
    }
}
```

Recommendation: Pass the `_cfg` into the `assertValidSharedLockbox` function to ensure that the deployed lockbox is using the expected config.

3.2.3 `ORACLE` checks are missing in `assertValidMipsVm`

Severity: Low Risk

Context: `MIPS64.sol#L93-L101`, `StandardValidatorUtils.sol#L182-L201`

Description: The constructor of the `MIPS64.sol` contract initiates the `STATE_VERSION` and the `ORACLE` when the contract is constructed. The associated assertion has a check for the `STATE_VERSION` but is missing the `ORACLE` check. This can result in the contract being setup with the incorrect oracle, which would not be caught with the assertion function.

Recommendation: Add an assertion to check if the `ORACLE` is correct.

3.2.4 Renounced SystemConfig owner blocks normal OPCM upgrades

Severity: Low Risk

Context: `SystemConfig.sol#L223-L225`

Description: `SystemConfig` inherits `OwnableUpgradeable` and does not override `renounceOwnership`. Therefore, the current `SystemConfig.owner()` can call the inherited function and set the owner to `address(0)`.

That zero owner breaks normal OPCMv2 upgrades. OPCMv2 loads the current `SystemConfig.owner()` into `FullConfig.systemConfigOwner` and passes that value back into `SystemConfig.initialize`. If ownership has been renounced, the initializer receives `address(0)`. It then calls `transferOwnership(_owner)` and OpenZeppelin rejects the zero address.

This means a `SystemConfig` whose ownership was renounced can block standard OPCMv2 upgrades even when the chain `ProxyAdmin` owner is still valid. Recovery requires a non-standard proxy-admin action or a special migration instead of the intended audited release process.

Recommendation: Override `renounceOwnership` in `SystemConfig` and make it revert, or use an ownership primitive that does not expose irreversible renounce behavior for this contract. If zero-owner `SystemConfig` instances must be recoverable, add an explicit OPCMv2 recovery instruction that can set a new nonzero owner with strict validation.

3.2.5 Guardian address check should be added to the `assertValidSuperchainConfig` function

Severity: Low Risk

Context: `StandardValidatorUtils.sol#L106-L131`, `StandardValidatorUtils.sol#L115`

Description: The super `SuperchainConfig` contract sets guardian on the `initialize` function. A check should be added on the associated assertion function should be added, to ensure the contract was setup with the correct guardian address.

Given the guardian has significant control over the contract, by way of pausing, unpausing, extending the pause duration — a `SuperchainConfig` setup with the incorrect value can be catastrophic for the functioning of the contract.

Recommendation: Add an assertion to ensure that the `SuperchainConfig` guardian is equal to a specific address. checking equivalence on an address should also warn of cases in which the guardian has been set to `address(0)` or unintentionally renounced.

3.2.6 Missing check on bridge version `assertValidOptimismMintableERC20Factory`

Severity: Low Risk

Context: `StandardValidatorUtils.sol#L203-L224`, `StandardValidatorUtils.sol#L220-L222`

Description: The associated assertion function checks to ensure that the bridge address is correct. Technically speaking, the address may be correct, but the bridge version may still be out of date. Consider checking the bridge version, and `systemConfig` in this assertion as well.

Recommendation: Add a check to ensure the bridge version is also the same, to ensure that the bridge has been updated and migrated correctly.

3.2.7 Validator misses portal `AnchorStateRegistry` check

Severity: Low Risk

Context: `StandardValidatorUtils.sol#L256-L280`

Description: The validators do not directly validate the `AnchorStateRegistry` configured on each portal.

`StandardValidatorUtils.assertValidOptimismPortal()` checks `portal.disputeGameFactory()`, but `OptimismPortal2` derives that value from `portal.anchorStateRegistry().disputeGameFactory()`. A wrong registry can return the expected DGF and still fail the normal ASR checks for implementation, proxy admin, `SystemConfig`, or registry behavior.

The migration validator has the same missing check. It discovers the shared ASR from the first portal, checks that every portal reports the same ASR, and checks that this ASR has a super respected game type. It does not require the portal ASR to match the ASR encoded in the validated shared dispute-game args.

This is a release-gate validation gap. It requires bad or malicious deployment/migration configuration because unprivileged users cannot install the wrong ASR. If that configuration is accepted, the portal's withdrawal finality checks use an unchecked registry.

Recommendation: Validate each portal's `anchorStateRegistry()` before certifying the portal or migration.

For standard validation, fetch `IAncorStateRegistry asr = _portal.anchorStateRegistry()` and run the existing ASR checks against it. For migration validation, also require the ASR used by each portal to match the ASR decoded from the validated shared dispute-game args.

3.2.8 Validator omits critical `SystemConfig` checks

Severity: Low Risk

Context: `OPContractsManagerStandardValidator.sol#L271-L301`

Description: `assertValidSystemConfig` validates the `SystemConfig` implementation, gas settings, resource config, operator fee fields, and `SuperchainConfig`, but it omits several live `SystemConfig` fields that deployment assertions and runtime bridge logic treat as expected chain configuration.

The omitted fields include `SystemConfig.owner()`, `batcherHash()`, `unsafeBlockSigner()` and `l2ChainId()`. The owner is a live administrative role that can change the batcher hash, unsafe block signer, L2 gas limit, fee scalars, EIP-1559 parameters and minimum base fee through `onlyOwner` setters.

Recommendation: Add expected `SystemConfig` authority, sequencing and chain-identity fields to standard validation and reject mismatches. At minimum, validate:

- `SystemConfig.owner()` against the expected `systemConfigOwner`;
- `SystemConfig.batcherHash()` against the expected batcher address encoded as `bytes32(uint256(uint160(expectedBatcher)))`;
- `SystemConfig.unsafeBlockSigner()` against the expected unsafe signer;
- `SystemConfig.l2ChainId()` against the `l2ChainID` being validated.

If the expected owner can differ from `l1PA0Multisig`, add a dedicated validation input or override field instead of silently accepting any owner. For migrated chains, validate each chain's `SystemConfig.l2ChainId()` against the migration manifest and ensure super-root proof validation uses that same value consistently. Consider also validating `batchInbox()` and `startBlock()` because the deployment assertions already treat those fields as expected `SystemConfig` state.

3.2.9 Assertion checks should ensure that owners of the `disputeGameFactory` and `proxyAdmin` are sound

Severity: Low Risk

Context: `OPContractsManagerV2.sol#L957-L974`, `StandardValidatorUtils.sol#L119-L131`, `StandardValidatorUtils.sol#L226-L254`

Description: The `OPContractsManagerV2.sol` implements the contracts such that the owner of the `disputeGameFactory` and `proxyAdmin` contracts are `addr(this)` for the duration of the upgrade/migration, allowing itself to make privileged calls, required to set-up the system.

Once the deployments are complete, this code checks if it's being called for the initial deployment, and if so, it changes the ownership of these two contracts to the `proxyAdminOwner`.

Notably, the related assertion functions (`assertValidDisputeGameFactory`) should have checks on factory owner to ensure that it is also set to the same admin as specified in the config. A check exists in this function, but only that the owner is equivalent to the `_l1PA0Multisig`, and not that the `_sysCfg` is also equal to the correct address. The `assertValidProxyAdmin` also implements this same logic.

Recommendation: In the case of asserting a valid dispute game, add a check to ensure that `_l1PA0Multisig == _sysConfig.owner()`; and in assertion proxy admin, add a sysconfig to the args list before implementing the same check.

3.2.10 Unauthorized root-block challenge wins bond

Severity: Low Risk

Context: `PermissionedDisputeGame.sol#L37-L68`

Description: `PermissionedDisputeGame` gates `step` and `move` with `onlyAuthorized`, but it does not override the inherited `challengeRootL2Block` function. The inherited function is still externally callable by anyone. When its proof checks pass, it records `l2BlockNumberChallenger = msg.sender`.

During root subgame resolution, `FaultDisputeGame.resolveClaim` treats that recorded address as the winner of the root claim bond. Consequently, any account can submit the root L2 block number challenge for a permissioned game and become the recipient of the proposer's root bond. The account does not need to be the configured proposer or challenger.

This does not allow an invalid root claim to finalize. The challenge still requires a valid output-root proof and a block header whose number differs from the claimed L2 block number. The problem is the mismatch with the permissioned game model: this game is meant to restrict dispute participation to the configured proposer and challenger, but this challenge can be taken by anyone. As a result, the configured challenger does not exclusively control the root-block challenge, and an arbitrary caller can capture the bond reward when this special challenge is available.

Recommendation: Override `challengeRootL2Block` in `PermissionedDisputeGame` and apply the intended permission check. If both permissioned roles are meant to perform this challenge, use `onlyAuthorized`. If the bond reward should be reserved for the challenger role, require `msg.sender == challenger()`.

3.3 Informational

3.3.1 ASR cutovers can leave old games valid

Severity: Informational

Context: `AnchorStateRegistry.sol#L151-L170`

Description: `AnchorStateRegistry` lets the guardian change the respected game type and the retirement cutoff only through two separate calls. `updateRetirementTimestamp()` retires games created at or before the current timestamp, while `setRespectedGameType()` changes which game type new games will snapshot as respected.

That split leaves no atomic way to retire old games and switch the respected game type. If the guardian updates the retirement timestamp first, any public caller can create a game of the old respected type in a later block before the respected type is changed. That game records `wasRespectedGameTypeWhenCreated == true`. It is also newer than `retirementTimestamp`, so `isGameRetired()` returns false. After the guardian switches to the new game type, `isGameRespected()` still accepts the old game because it only reads the creation-time snapshot.

Consequently, the old game can later resolve `DEFENDER_WINS`, pass `isGameClaimValid()` and call `setAnchorState()`. The same predicate is used by `OptimismPortal2.checkWithdrawal()`, so withdrawal finalization can also keep trusting a game type that the guardian meant to retire during the cutover.

This matters during emergency or proof-system transitions. If the old game type is being retired because it is unsafe, unavailable or incompatible with the new proof mode, a public caller can preserve one old game past the cutoff and later use it as an anchor or withdrawal root if it is not successfully challenged.

Recommendation: Add an atomic guardian operation that sets the new respected game type and retirement timestamp in the same call. The function should retire games created at or before the transaction timestamp and switch `respectedGameType` before any later transaction can create another old-type game with a respected snapshot.

For upgrade-driven cutovers, use the same atomic operation or add an explicit initializer mode that advances the retirement timestamp when the respected game type changes.

3.3.2 Portal misses chain-ID check for non-super game roots

Severity: Informational

Context: [OptimismPortal2.sol#L415-L423](#)

Description: `OptimismPortal2.proveWithdrawalTransaction` binds super-game roots to the portal's L2 chain ID by calling `rootClaimByChainId(systemConfig.l2ChainId())`. It does not do the same for non-super dispute games. For those games, it reads `rootClaim()` directly.

That skips a check that already exists on single-chain games. `FaultDisputeGame.rootClaimByChainId` reverts with `UnknownChainId` when the requested chain ID does not match the game's immutable `l2ChainId()`. Since the portal does not call that function for non-super games, it never independently confirms that the game root belongs to the chain configured in this portal's `SystemConfig`.

`AnchorStateRegistry.isGameClaimValid` does not close this gap. It checks that the game is registered, proper, respected, finalized and resolved in favor of the defender. It does not check that the game was created for this portal's L2 chain ID.

This is mainly a missing fail-closed check. In a normal deployment, OPCM and the standard validation process should put the correct L2 chain ID into the non-super game implementation arguments. An unprivileged user cannot normally choose that value. The likelihood of a potential issue, though, is quite low as this risky state requires a privileged factory registration, migration or owner-controlled upgrade that leaves this portal respecting a non-super game type configured for the wrong chain. Once that bad configuration exists, any proposer can create a game for the wrong chain. They can then prove a withdrawal against an output root from that other chain, wait for the game to resolve, and finalize through this portal. The portal therefore relies on deployment-time correctness for a consensus-critical value that the game interface can verify at runtime.

Recommendation: Make withdrawal proving chain-bind every dispute game before accepting its root claim. In `proveWithdrawalTransaction`, call `disputeGameProxy.rootClaimByChainId(systemConfig.l2ChainId())` for non-super games as well as

super games. A single-chain game with the wrong `l2ChainId()` should fail before the withdrawal proof is recorded.

Fix `ZKDisputeGame.rootClaimByChainId` in the same change so it matches `FaultDisputeGame` and reverts unless the requested chain ID equals `l2ChainId()`. Keep the existing deploy-time and validator checks, but do not depend on them as the only protection. The portal should reject a wrong-chain game even if that game type is accidentally registered as respected.

3.3.3 Add check on `START_BLOCK_SLOT` in `assertValidSystemConfig`

Severity: Informational

Context: `OPContractsManagerStandardValidator.sol`#L272

Description: In the constructor, the `START_BLOCK_SLOT` is set to `type(uint256.max)`; and in the `initialize`, it is set to `block.number`. The assertion function should check equivalence on the correct `block.number` being used.

Recommendation: Add an internal require to check the correct `START_BLOCK_SLOT`.